# Reporte de Práctica: Desarrollo Backend con Django, DRF y JWT

Proyecto: **Tienda Online API** | Autor: Ricardo (GitHub: @Zerkantz)

| | |
|---|---|
| **Estudiante:** Ricardo | **Repositorio GitHub:** https://github.com/Zerkantz/Django |
| **Tema del Proyecto:** Tienda Online (Comercio Electrónico) | **Tecnologías:** Django 5.2, DRF, SimpleJWT, Swagger (drf-yasg), MySQL |
| **Campo extra en Usuario:** telefono | **Tiempo de vida Access Token:** 2 horas (ACCESS_TOKEN_LIFETIME) |

## 1. Objetivo y Adaptaciones Realizadas

Se implementó y fortaleció el backend guiado por el manual *'Desarrollo Full Stack con Django y ReactJS'* (págs. 5 a 12), adaptándolo a una temática personalizada y solventando fallas de seguridad del diseño original. Partiendo del proyecto de la Unidad 1 (Django + MySQL + DRF + Catálogo de Productos), se incorporaron los siguientes cambios:

- **Temática y Nombre Propio:** Se definió el proyecto como **Tienda Online** (reemplazando el nombre 'restaurante'), utilizándolo como núcleo del backend y en el encabezado de Swagger OpenAPI.
- **Modelo de Usuario Personalizado:** Modelo users.User heredando de AbstractUser con USERNAME_FIELD = 'email' para login con correo.
- **Campo Extra Personalizado (telefono):** Añadido al modelo, reflejado en el panel Admin de Django, en el serializador UserSerializer y en los esquemas OpenAPI de Swagger.
- **Tiempo de Vida del Token JWT:** Configuración explícita de ACCESS_TOKEN_LIFETIME = timedelta(hours=2) y REFRESH_TOKEN_LIFETIME = timedelta(days=7) en settings.py.

## 2. Identificación y Corrección de Fallas de Seguridad

El código base del manual presentaba dos fallas de seguridad críticas que fueron identificadas y corregidas:

| Falla en Manual Original | Impacto / Vulnerabilidad | Corrección Implementada |
|---|---|---|
| **1. Fuga de Contraseña:** El UserSerializer incluía el campo password sin write_only=True. | Toda respuesta GET (incluido /api/v1/auth/me/) exponía el hash PBKDF2 de la contraseña del usuario en texto plano. | Se configuró extra_kwargs = {'password': {'write_only': True}}. La contraseña puede recibirse al crear o actualizar, pero **nunca** se serializa en respuestas. |
| **2. Sobrescritura de Contraseña:** En partial_update del viewset: else: data['password'] = request.user.password | Al actualizar un usuario sin enviar contraseña, el manual le asignaba la contraseña encriptada del admin peticionario. El usuario perdía su contraseña original. | Se eliminó dicha asignación. En partial_update y update solo se altera la contraseña si viene explícitamente en el payload con un nuevo valor. |

# 3. Evidencias Gráficas del Sistema

## 3.1 Documentación Interactiva Swagger (drf-yasg)

Esquema OpenAPI interactivo con título **'Tienda Online API v1'**, exponiendo los endpoints de Usuarios, Autenticación JWT, Categorías y Productos:

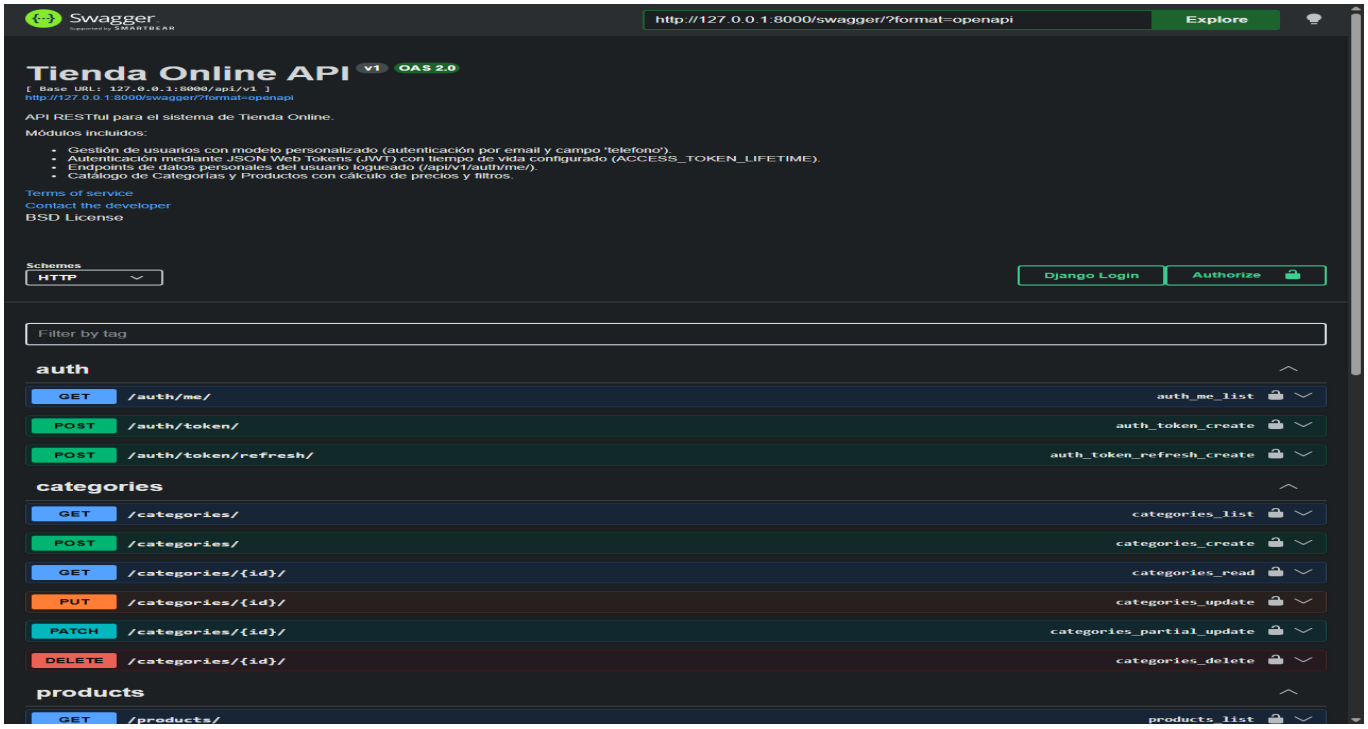



*Figura 1: Interfaz Swagger UI en http://127.0.0.1:8000/swagger/ con los módulos organizados.*

## 3.2 Panel de Administración de Django con Campo Extra 'Teléfono'

Visualización del modelo de usuario en el Django Admin, destacando la columna personalizada **'TELÉFONO'** y el inicio de sesión con email:

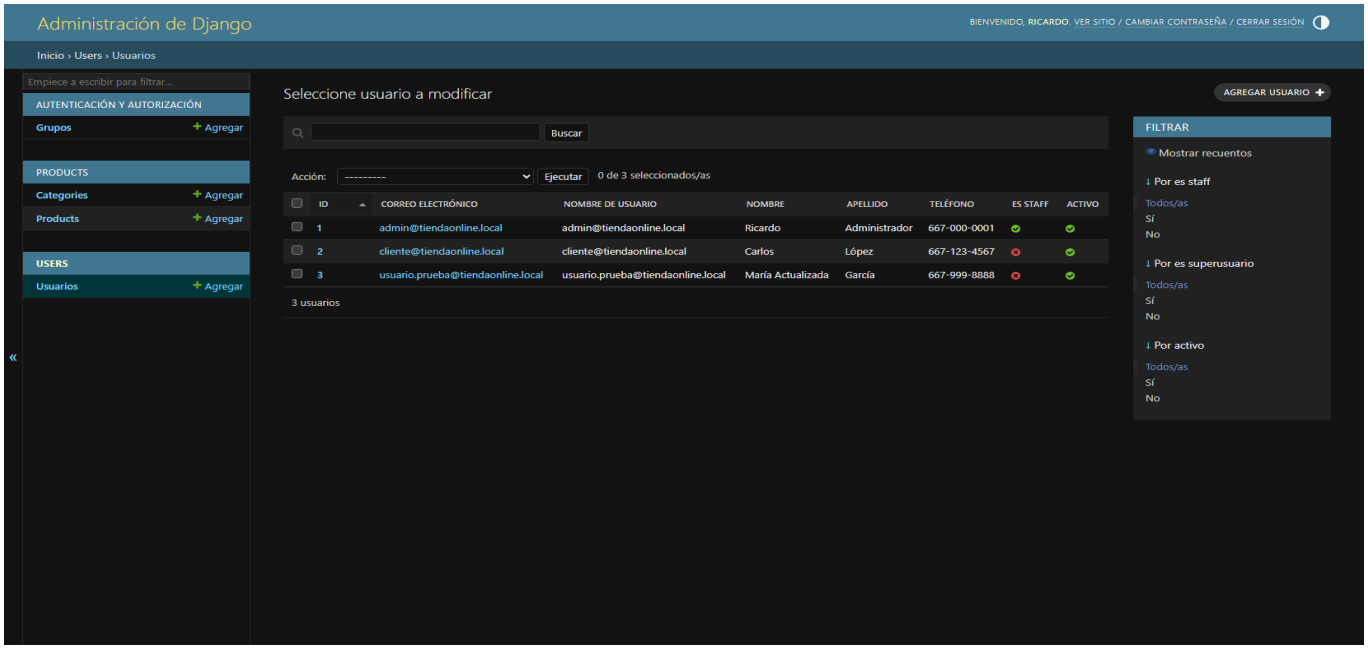



*Figura 2: Django Admin con el modelo de usuarios personalizado y el campo extra telefono.*

### 3.3 Obtención del Token JWT (/api/v1/auth/token/)

Inicio de sesión por email y contraseña. Se comprueba la entrega de access (vida útil de 2 horas) y refresh token:

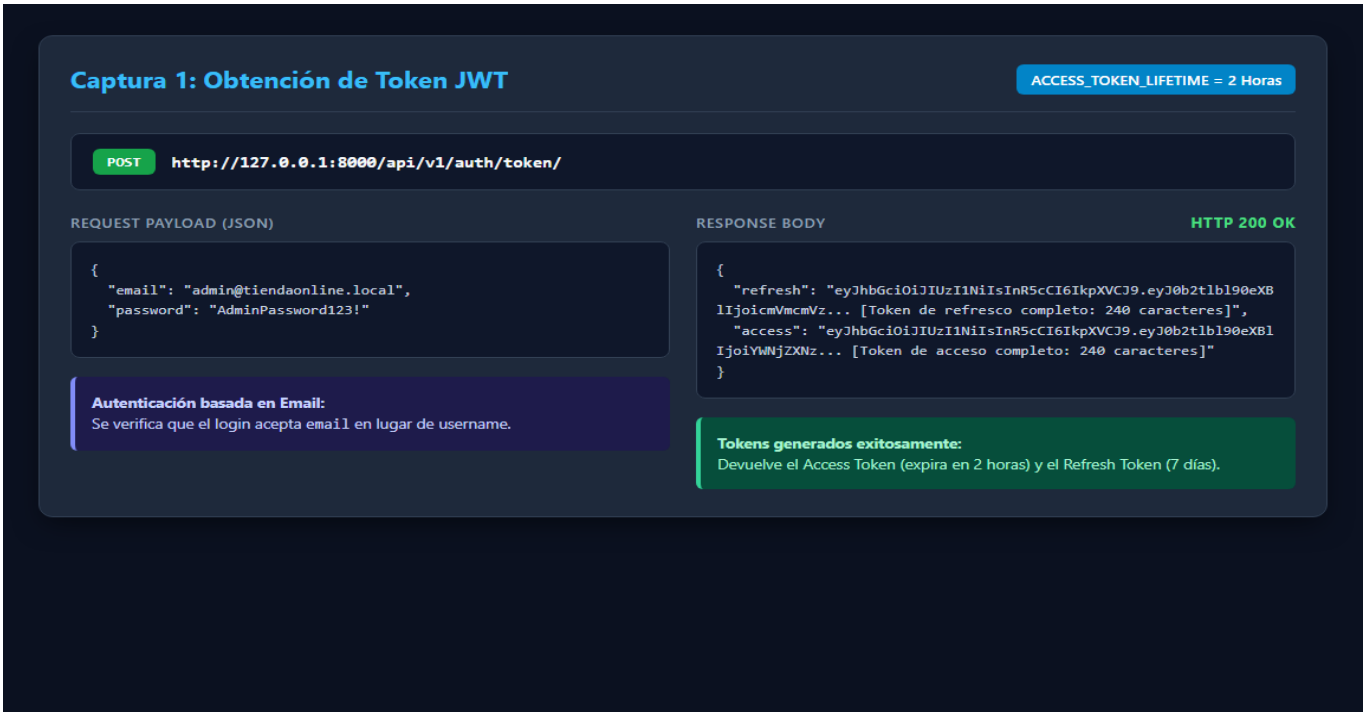



*Figura 3: Petición POST a /api/v1/auth/token/ con email retornando tokens de autenticación.*

### 3.4 Consulta de Datos Personales (/api/v1/auth/me/)

Petición GET autenticada mediante cabecera Authorization: Bearer <token>. Se verifica la presencia del campo extra telefono y la exclusión absoluta del campo password:

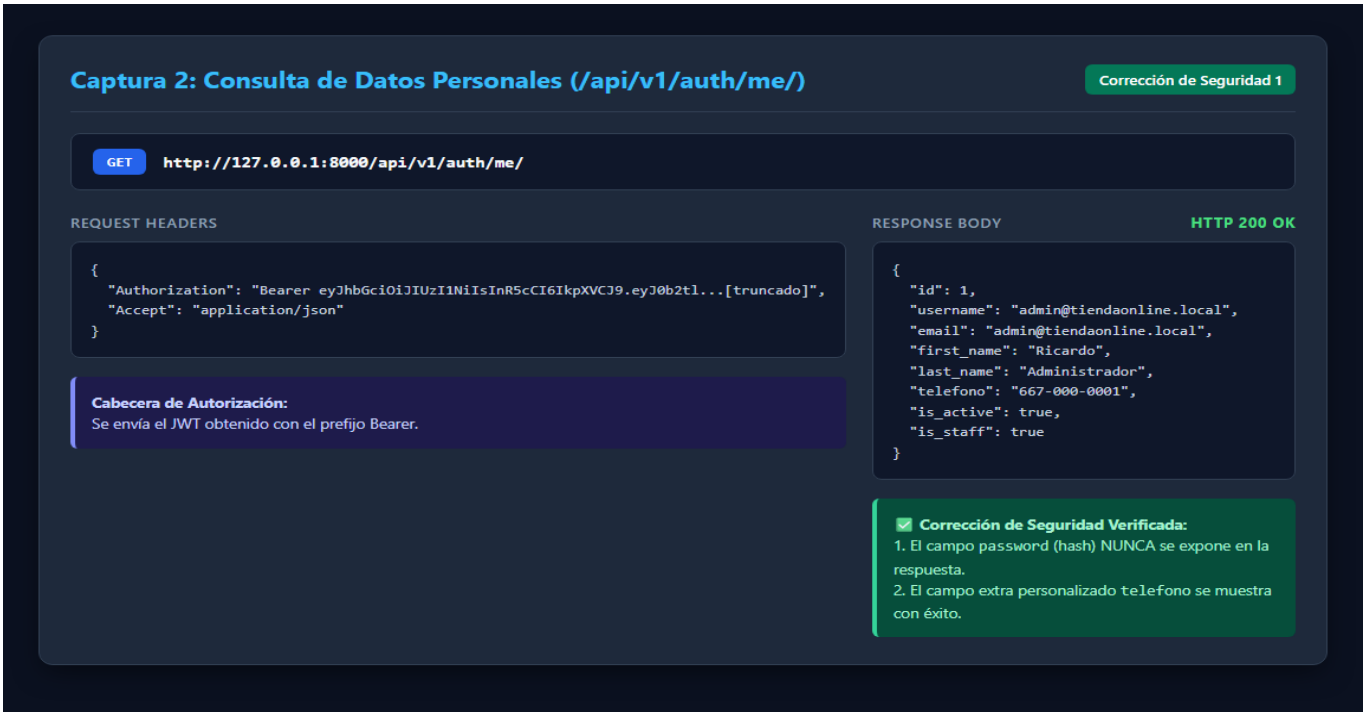



*Figura 4: Endpoint /api/v1/auth/me/ con datos del usuario. Corrección de seguridad verificada (sin hash de password).*

### 3.5 Prueba Crítica de Seguridad: Edición sin Contraseña

Demostración experimental de que al editar un usuario sin enviar contraseña (PATCH), se conservan sus datos previos y **NO** se le asigna la contraseña del administrador:

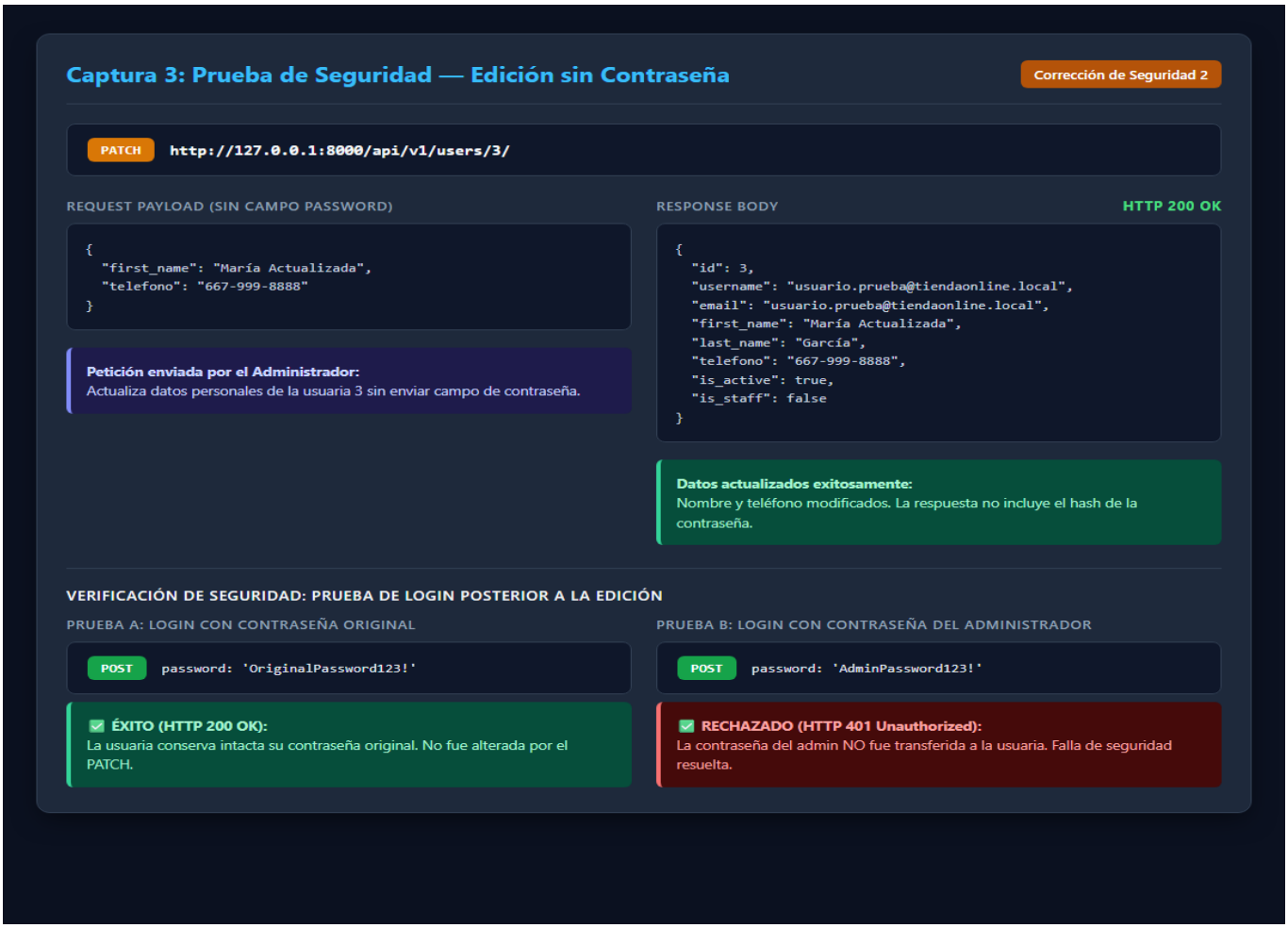



*Figura 5: PATCH sin contraseña -> Login con contraseña original: 200 OK -> Login con contraseña de admin: 401 Rechazado.*

## 4. Verificación Automatizada con Django Tests

Se construyeron 16 pruebas unitarias e integrales (100% aprobadas) para garantizar la calidad del software:

| Módulo de Pruebas | Aspectos Validados | Resultado |
|---|---|---|
| UserModelTestCase | Creación con email, teléfono y superusuario con permisos. | **PASÓ (OK)** |
| SecurityFixesTestCase | Validación de no exposición de password y preservación en PATCH. | **PASÓ (OK)** |
| JWTAuthenticationTestCase | Emisión con email, expiración en 7200 s y consumo en /auth/me/. | **PASÓ (OK)** |
| CategoryTestCase | Creación y listado de categorías en la API REST. | **PASÓ (OK)** |
| ProductTestCase | CRUD completo y endpoint de precio final con descuento. | **PASÓ (OK)** |

## 5. Conclusiones y Consideraciones Finales

El desarrollo de esta práctica permitió consolidar la implementación de arquitecturas de autenticación desacopladas con JSON Web Tokens (JWT) en Django REST Framework, integrando un modelo de usuario personalizado basado en correo electrónico y extendido con atributos propios de la aplicación (telefono).

Un aprendizaje fundamental del ejercicio fue el rigor en la auditoría de seguridad del código fuente: los manuales y guías de introducción a menudo omiten validaciones estrictas o introducen atajos metodológicos que, de aplicarse en producción, expondrían credenciales confidenciales o vulnerarían cuentas de usuarios. Corregir estas dos debilidades (restringir el hash a write_only y preservar la integridad del password en actualizaciones parciales) eleva el proyecto a estándares profesionales de seguridad en software.

**DATOS DE ENTREGA Y REPOSITORIO DEL PROYECTO**

**Enlace Oficial del Repositorio de GitHub:**
**https://github.com/Zerkantz/Django**

**Elementos clave verificados en el repositorio:**
• **.gitignore:** Excluye entornos virtuales (venv/, .venv/), bases de datos locales (db.sqlite3), archivos de caché de Python (__pycache__/, *.pyc) y temporales.
• **requirements.txt:** Dependencias completas en UTF-8 (Django 5.2, DRF 3.18, djangorestframework-simplejwt, drf-yasg, mysqlclient, reportlab, etc.).
• **users app:** Modelo User personalizado, endpoints de autenticación JWT (/api/v1/auth/token/, /api/v1/auth/token/refresh/), endpoint de datos propios (/api/v1/auth/me/) y CRUD administrativo de usuarios.
• **products app:** Integración completa del catálogo de la Unidad 1 (Categorías, Productos y Cálculo de Precio Final).
• **Documentación interactiva:** Swagger UI disponible en /swagger/ y ReDoc en /redoc/.